

SmartBancs App

Reto Técnico NextGen Engineer

Tata Consulting Services · TRAINEE IT

10,000 TPS

Concurrencia

< 2 seg

Por transacción

NestJS + Python

Gemini AI

Recomendaciones

El Desafío

Restricciones críticas que definen la arquitectura

10,000 TPS

Alta concurrencia con procesamiento paralelo sin degradación del servicio

< 2 segundos

Latencia máxima por transacción, incluyendo persistencia en BD

Core Bancs legado

No puede recibir el volumen directo. Requiere ACL y sincronización asíncrona

IA no bloqueante

Las recomendaciones no pueden retrasar el flujo transaccional bajo ningún escenario

Race Conditions

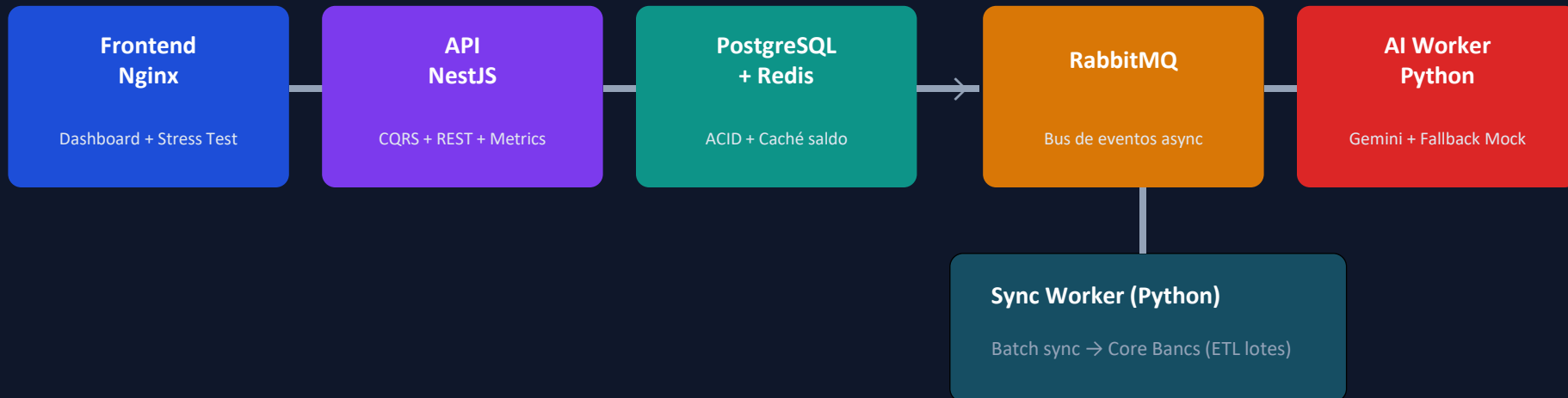
Evitar inconsistencias de saldo con transferencias simultáneas sobre la misma cuenta

Observabilidad

Logs, métricas y trazas para detectar, diagnosticar y resolver incidentes

Arquitectura del Sistema

Event-Driven + CQRS + Stack polígota (NestJS + Python)



Flujo síncrono (< 2s) → retorna OK al cliente

Flujo async (IA + Bancs) → no bloquea al cliente

Stack Tecnológico

Cada herramienta elegida con justificación explícita

NestJS / TypeScript

@nestjs/cqrs nativo · Módulos explícitos · ~30K req/s

PostgreSQL 16

MVCC · ACID · SELECT FOR UPDATE · pg_stat_statements

Redis 7

Write-through · Saldo < 1ms · allkeys-lru 256MB

RabbitMQ 3.12

Desacoplamiento async · UI admin · Migración a Kafka documentada

Python / Gemini AI

Ecosistema ML superior · google-generativeai · Fallback mock

Prometheus + Grafana

Métricas de TPS, latencia p99, errores · Alertas configuradas

CQRS + Control de Concurrency

La arquitectura que previene race conditions y deadlocks

CQRS con @nestjs/cqrs

① Controller

POST /transactions → CommandBus.execute()

② ProcessTransactionCommand

DTO empaquetado con traceId único

③ CommandHandler

Lógica de negocio + QueryRunner ACID

④ SELECT FOR UPDATE

UUIDs ordenados → sin deadlocks cruzados

⑤ EventBus.publish()

TransactionCompletedEvent → async AI

⑥ 200 OK

< 500ms promedio en carga normal

Anti-Deadlock: Orden Canónico

El problema:

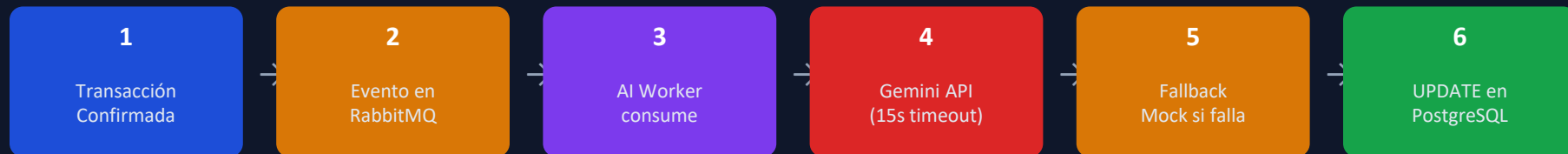
Hilo A bloquea cuenta X → espera Y
Hilo B bloquea cuenta Y → espera X
→ DEADLOCK circular

La solución:

```
const [first, second] =
  [accountFrom, accountTo].sort();
// Ambos hilos bloquean el mismo
// UUID primero → uno espera,
// el otro completa.
```

Inteligencia Artificial

Recomendaciones financieras async con Gemini + fallback resiliente

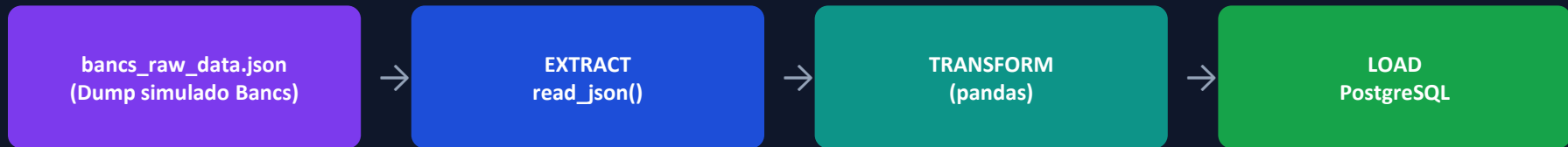


Modelo usado:	gemini-flash-latest (baja latencia en texto corto)
Prompt:	Contexto SmartBancs + monto. Max 2 oraciones de recomendación
Resiliencia:	Timeout 15s → 4 mensajes mock predefinidos como fallback
Métricas:	ai_requests_total{status="success error"} en Prometheus
Escalado:	Múltiples consumers de la misma cola (horizontal scaling)

Integración Bancs + ETL

Anti-Corruption Layer que protege el core legado

El core Bancs NUNCA recibe 10,000 TPS directamente. El sistema acumula y consolida antes de sincronizar.



Transformaciones aplicadas:

- ① `drop_duplicates(subset=["transaction_ref"])` - elimina duplicados del legado
- ② `dropna(subset=["account_from", "amount"])` - descarta registros incompletos
- ③ `to_datetime(utc=True)` - estandariza fechas
- ④ `str.replace(r"^\d.", "")` - limpia montos con caracteres extraños
- ⑤ Inserta en `bancs_raw_transactions` vía SQLAlchemy (mode=append)

Observabilidad

Tres pilares: Logs · Métricas · Trazas

Logs

- NestJS Logger estructurado con contexto
- TraceID propagado en cada operación
- Python AI Worker: logging.INFO/ERROR
- Campos: traceId, txId, amount, durationMs

Métricas

- Prometheus: GET /metrics cada 15s
- smartbanks_transactions_total (counter)
- smartbanks_transaction_duration_seconds (hist)
- smartbanks_ai_requests_total (counter)

Trazas

- TraceID (UUID v4) por cada request
- Propagado: API → PostgreSQL → RabbitMQ → AI
- Dashboard Grafana: TPS, p99, errores
- RabbitMQ UI: mensajes pendientes en cola

⚠️ Alertas: p99 > 1.5s (warning) · p99 > 2s (critical) · error_rate > 1% (PagerDuty) · queue > 10K mensajes (escalar AI Worker)

Incidente Simulado

Deadlock en pico de quincena · 47 minutos de degradación

47 min

Duración

5,200ms

Latencia p99

7.8%

Error rate

340 / 10K

TPS real



14:30

Pico inicia. TPS: 1.2K → 9.8K en 4 min



14:33

Prometheus alerta p99 > 1.5s. Primer deadlock en pg_stat_activity



14:42

Diagnóstico: 23 tx bloqueadas. Full table scan identificado con EXPLAIN ANALYZE



14:45

pg_terminate_backend() en 3 PIDs bloqueantes. Latencia baja a 2.1s



14:55

CREATE INDEX CONCURRENTLY en producción. Circuit breaker + Redis activo



15:17

Sistema estable. p99 = 280ms. Error rate = 0%. Incidente cerrado

Demo del MVP

`docker compose up --build` · Un comando, entorno completo

01 · Arranque

`docker compose up --build`

10 contenedores en ~90 segundos

02 · Dashboard

`http://localhost:8080`

UI glassmorphism · Crear cuentas · Transferir

03 · Stress Test

`50 req concurrentes`

Simular pico transaccional desde el botón en el frontend

04 · API REST

`POST /transactions`

`curl` / Postman con `accountFrom`, `accountTo`, `amount`

05 · Grafana

`http://localhost:3001`

TPS, latencia p99, errores en tiempo real

06 · RabbitMQ UI

`http://localhost:15672`

Cola `tx.completed` · Mensajes pendientes

Gaps Identificados y Próximos Pasos

Transparencia sobre lo pendiente y ruta de evolución

sync-worker

Gap

No implementado

Estado

Sin sync real con Bancs

Impacto

→ Consumer RabbitMQ → API Bancs + retry

Acción correctiva

Tests

Gap

Directorio vacío

Estado

Sin cobertura de tests

Impacto

→ Jest unit tests ProcessTransactionHandler

Acción correctiva

seed.sql

Gap

⚠ Solo 2 cuentas

Estado

Datos de prueba limitados

Impacto

→ 10+ cuentas con distintos balances

Acción correctiva

JWT Auth

Gap

Sin auth

Estado

Endpoints expuestos

Impacto

→ NestJS Guard con JWT Bearer token

Acción correctiva

PgBouncer

Gap

Pendiente

Estado

Sin pool management

Impacto

→ max_conn=200, pool_size=20 por servicio

Acción correctiva

SmartBancs App

github.com/ErickxBa/SmartBancs_App_EB

Erick Ballas · Septiembre 2026